

Lets understand the problem first:

To create an immutable class, which is required as simply data carrier (like a POJO), we need to write below code:

```
User userObj1 = new User("name1", 28);  
userObj1.getName();  
userObj1.getAge();
```

We already saw, how Lombok helps in reducing some of these boiler code like:

- automatic generation of getters and setters via **@Getter** and **@Setter**
- automatic generation of equals and hashCode method via **@EqualsAndHashCode**
- automatic generation of constructors via **@AllArgsConstructor**
- automatic generation of toString()

method via **@ToString**

- automatic generation of Immutable class via **@Value**

One thing we can easily notice is: **Lots of Boiler code** (just for basic immutable POJO class)



We have already covered it in depth, if there is any doubt with "how Lombok works", kindly check it out.

And that's where **Java16 Records** comes into the picture.

But wait:

Then why need "Record"?

Will come back to this question, after we understand, how does **Records works:**

Records:

- **It help us to create immutable class in a short way.**
- **It is mostly designed to reduce boiler code for data carrying classes (like POJO)**

```
public final class User {  
  
    private final String name;  
    private final int age;  
  
    public User(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
  
    public String getName() {  
        return name;  
    }  
}
```

```
}
```

```
public int getAge() {  
    return age;  
}
```

```
@Override  
public boolean equals(Object o) {  
    /* equals logic here */  
}
```

```
@Override  
public int hashCode() {  
    /* hashCode logic here */  
}
```

```
@Override  
public String toString() {  
    /* toString logic here */  
}  
}
```

```
public final class User {
```

```
    private final String name;  
    private final int age;
```

```
    //no setter methods, initialized via constructor only  
    public User(String name, int age) {  
        this.name = name;  
        this.age = age;
```

```
}
```

//only getter method is available

```
public String getName() {  
    return name;  
}
```

```
public int getAge() {  
    return age;  
}
```

//generally in POJO like classes, we also provide, equals, hashCode and toString methods

```
@Override  
public boolean equals(Object o) {  
    /* equals logic here */  
}
```

```
@Override  
public int hashCode() {  
    /* hashCode logic here */  
}
```

```
@Override  
public String toString() {  
    /* toString logic here */  
}  
}
```

Syntax: *record YourRecordName*(*Type* field1, *Type* field2, ...) { }
public record User (String name, int age)
{ }

User.class

javac User.java

- generates "private final" fields, generally called "record components"
- We can not add more instance fields.
- static fields are allowed, as they don't belongs to an individual instance.
So each User Object is still immutable.

all instance fields must be defined in record header only, that's why sometime its called "Transparent data carrier"

as just by seeing record, we can find, what this record is carrying.

- **record** keyword is equivalent to "final class"
- **record** access specifier rules are similar to normal class.
- All **record** class by-default extends "java.lang.Record" class implicitly.
Why? because Java framework can only identify "User" is a record or not via superclass only, internally framework code does like this:

getSuperclass()==java.lang.Record.class

- That's why, extends are not allowed on Record (as in Java, we know that we can not extend from more than 1 class (multiple inheritance is not supported))

- But we can implement more than one interfaces like any other normal class.

User.java

Output:

- Automatically generates "canonical constructor", means constructor that takes all record components (fields) in order.
- We can override the canonical constructor if want.
- We can have "Compact Constructor", which is a shorthand form of canonical constructor. Compiler implicitly assumes the parameters are the record components in the declared order.

The above constructor can be written as:

- **We can have more constructors with different parameters list, but directly or in-directly they must delegate the call to canonical constructor.**

Why? Because canonical constructor gives guarantee that all fields(components) will get initialized.

If this is not mandatory then there is a chance that some field might get uninitialized.

- **By default, Implicit declaration of canonical constructor has the same accessibility as the record class.**

But if we override the canonical constructor, we can not restrict the access level.

means:

public record -> public canonical constructor

```
public record User(String name, int age) {
```

```
// Canonical constructor must be public (because record is public)
public User(String name, int age) {
    this.name = name;
    this.age = age;
}
}
```

record(package level) -> either same package level or public canonical constructor both are allowed

```
record User(String name, int age) {
    // Record is package-private → constructor also package-private
    User(String name, int age) {
        this.name = name;
        this.age = age;
    }
}
```

```
record User(String name, int age) {
```

```
// Record is package-private → canonical constructor is  
public  
public User(String name, int age) {  
    this.name = name;  
    this.age = age;  
}  
}
```

- **Make sure of doing Defensive Copying.**

Here, even though hobbies reference is immutable but the list it points too, its not.

Output:

So always make sure of doing defensive copying.

As internally copyOf method creates unmodifiable list.

If we try to update:

- **For every record component (fields) automatically generate a public accessor method.**
 - **They are named exactly as the component (field)**
 - **No setter method is created.**
 - **We can override these accessor methods.**
-
- **We can add more methods if required.**

- "equals", "hashCode" and "toString" method code is generated runtime when the class is loaded.

But we can Override them if required.

@Override

```
public boolean equals(Object o) { ... }
```

@Override

```
public int hashCode() { ... }
```

@Override

```
public String toString() { ... }
```

Nested Records:

If there is any doubt regarding, what is Nested Class, how to define and use it, kindly check the below topic first:

Nested records are very much similar to Nested Classes but with 1 minor change.

- **So in records, we can also created Nested records, which can be private/protected/public like Nested Classes.**
- **Only difference is, in record, only static Nested records are possible and non-static nested records are not possible.**

By default static only

And we can access it in similar way like we access nested classes.

1 more example:

And we can access it in similar way like we access nested classes.

Why only static records are allowed?

Sharing my view:

Records is a transparent data carrier. Means all its fields must be defined in the record header.

```
record User(String a) {  
  
    record Address(String b) { } // say  
    hypothetically its non-static  
}
```

```
private final User userObj; // hidden reference
```

Also since we assumed that nested record is non-static, means it can access parent class fields, so internally Address contain reference of User:

This **violates the promise** of records:

- Now the state of nested record Address is **not fully declared** in "*record Address(String b)*".

- It carries extra, invisible state (userObj).
- That breaks the **transparency** principle.

Local Records:

Similar to Local Class, we can also create Local records, means records within a block like method block, while loop block, if condition block etc..

Its access modifier is similar to Local classes i.e. local record can not be declared public/private/protected.

Why? Because local record scope is limited to block, so generally giving access modifier does not make any sense.

Local record can not be initiated outside of the block.

Also there is no static Local records because static means something belongs to class, but local records scope to a block not class.

Output: